NARC ENGINE

NARC Engine — Technical Report

Architecture, Training Methodology, and Distinguishing Characteristics

Madushan Dissanayake and Claude AI

Technical Design Document · Version 1.0

Contents

Abstract

1. Introduction and design goals

2. Board representation and move generation

- 2.1 Bitboards
- 2.2 Magic bitboards
- 2.3 Move encoding and make/unmake
- 2.4 Legality, SEE, and verification

3. Search

- 3.1 Framework
- 3.2 Transposition table
- 3.3 Pruning and reductions
- 3.4 Move ordering, quiescence, and time

4. Evaluation: classical and NNUE

- 4.1 Classical (hand-crafted) evaluation
- 4.2 NNUE — efficiently updatable neural evaluation
- 4.3 The HalfKA experiment

5. Parallel search (Lazy SMP)

6. Training methodology

- 6.1 Data generation
- 6.2 Network training
- 6.3 Cloud pipeline
- 6.4 Generational self-play

7. Testing and validation methodology

8. Empirical results — nine generations

9. Distinguishing characteristics and novelty

10. Engineering and deployment

11. Limitations and future work

12. Conclusion

Appendix A — Key parameters

Appendix B — Source layout

Abstract

NARC is a grandmaster-strength UCI chess engine written from scratch in C++20. It combines a magic-bitboard move generator, a modern alpha-beta search with the full contemporary pruning and reduction suite, and a dual evaluation system: a tapered hand-crafted evaluation and an **NNUE neural network trained entirely on the engine's own self-play**, with no externally sourced data, code, or weights. The engine searches ~1M nodes/second single-threaded and scales to ~3M with Lazy SMP multithreading. This report documents every major technical component, the complete data-generation and training pipeline (local and cloud), the empirical testing methodology, the measured results across nine engine generations, and an honest assessment of what distinguishes NARC from other engines. The headline empirical findings: the classical build defeated Stockfish 17.1 (limited to 2600 Elo) 6–4; a generational self-play NNUE gained +61 Elo over its predecessor; and Lazy SMP added +137 Elo at two threads.

1. Introduction and design goals

NARC was built to a single specification — *play at grandmaster strength and run in any UCI chess GUI* — under three self-imposed constraints that shaped every decision:

1. **From scratch.** No engine code, no borrowed neural network, no external opening books or tablebases. Even the magic-bitboard constants are *searched for* at runtime rather than copied from published tables.
2. **Self-contained.** The final artifact is a single static executable with the evaluation network embedded inside it — no companion files required.
3. **Empirically gated.** Every change had to *prove* itself in engine-vs-engine matches before being kept. Improvements that looked good on paper but did not win games were rejected and documented.

The result is a ~2,000-line engine plus a complete, reproducible training pipeline. The remainder of this report describes how each part works and what the gating process revealed.

---

2. Board representation and move generation

2.1 Bitboards

The board state is held in **bitboards** — 64-bit integers with one bit per square. NARC keeps occupancy by colour (`byColor[2]`), occupancy by piece type (`byPiece[6]`), and a redundant `board[64]` mailbox for O(1) piece-type lookup. Set operations (attacks, intersections, piece counts) become single machine instructions (`AND` , `popcount` , `ctz`), and pieces are iterated branch-free with the `b &= b - 1` idiom.

2.2 Magic bitboards

Sliding-piece attacks (rook, bishop, queen) use **magic bitboards**, a perfect hashing scheme: the relevant-blocker occupancy is multiplied by a magic constant and shifted to form a collision-free index into a precomputed attack table. Distinctively, NARC **searches for its own magic numbers at startup** (random sparse candidates tested against every blocker subset via a carry-rippler walk, with an epoch trick to avoid re-zeroing the table) rather than hardcoding published constants. The two tables total ~107 KB and stay largely in cache; sliding attacks resolve in three operations (mask, multiply, shift).

2.3 Move encoding and make/unmake

Moves are packed into **16-bit integers** (6 bits from, 6 bits to, 2 bits promo piece, 2 bits flag). `make()` / `unmake()` mutate the board in place — never copying it — and record only the minimal `Undo` state needed to reverse the move. The **Zobrist hash key** is maintained incrementally by XOR: because XOR is its own inverse, every board edit is trivially reversible, and the key, the bitboards, and (later) the NNUE accumulator are all threaded through the same make/unmake pair as reversible incremental state.

2.4 Legality, SEE, and verification

Generation is pseudo-legal; legality is enforced by playing the move and testing whether the mover's king is attacked. **Static Exchange Evaluation (SEE)** uses the swap algorithm with X-ray re-resolution to score capture sequences for move ordering and pruning. Correctness was proven with **perft** (leaf-node counting): NARC matches all five standard test positions exactly — e.g. the start position to depth six yields 119,060,324 nodes — at roughly 25 million nodes/second.

---

3. Search

NARC's search is a modern **negamax alpha-beta** with the contemporary high-performance feature set. Alpha-beta returns the same move as full minimax while skipping provably irrelevant branches; with good move ordering its complexity falls from O(b^d) toward O(b^(d/2)), which is why ordering and the transposition table dominate performance.

3.1 Framework

- **Iterative deepening** — search depth 1, 2, 3, … reusing each iteration's transposition and ordering data; provides an anytime best move under time pressure.
- **Aspiration windows** — after depth 4, search a narrow window (±25 cp) around the previous score, widening geometrically on fail-high/low.
- **Principal Variation Search (PVS)** — the first (best-ordered) move is searched with a full window; the rest are verified with cheap zero-width windows and re-searched only if they unexpectedly beat alpha.

3.2 Transposition table

A power-of-two hash table (default 128 MB) memoizes positions by Zobrist key, storing best move, score, search depth, a static-eval cache, and a bound type (exact / lower / upper). For multithreading the entries became **race-tolerant**: each slot stores `key XOR data`, so a torn read or write under a data race fails the key check and is treated as a miss rather than corrupting play. Mate scores are stored relative to the root via ply adjustment.

3.3 Pruning and reductions

The search applies, in non-PV nodes and gated by depth and check status:

- **Null-move pruning** (adaptive reduction; disabled without non-pawn material to avoid zugzwang),
- **Reverse futility pruning** (static null move),
- **Razoring** (low-depth qsearch verification when the eval is hopeless),
- **Futility** and **late-move pruning** (skip late/quiet moves near the leaves),
- **SEE pruning** (skip losing captures, and very poor quiets via history),
- **ProbCut** (a capture that beats a margin at reduced depth ends the node),
- **Late move reductions** — log-log reduction table (`0.75 + ln(d)·ln(m)/2.25`), adjusted by PV status, the improving flag, killers, and history; reduced searches that beat alpha are re-searched at full depth (verification keeps LMR sound),
- **Internal iterative reduction** when no hash move is available,
- **Check** and **singular extensions** (with **multi-cut**): when the TT move is far better than all alternatives in a verification search, it is extended; when several moves beat beta, the node cuts.

3.4 Move ordering, quiescence, and time

Moves are ordered: TT move → good captures (MVV-LVA filtered by SEE) → killers → counter-move → quiet moves by combined history → losing captures. The **history stack** comprises butterfly history, **continuation history** (conditioned on the previous one and two moves), and **capture history**, all updated with a gravity rule that bounds their range and lets them adapt.

Quiescence search at the leaves resolves captures, promotions, and check evasions (with SEE and delta pruning) so the static evaluation is only ever applied to quiet positions, eliminating the horizon effect.

Time management computes a soft limit (don't start a new iteration past it) and a hard limit (interrupt immediately), scaled by **best-move stability** — a move confirmed across iterations consumes less of the budget.

---

4. Evaluation: classical and NNUE

NARC ships two evaluation functions and dispatches to NNUE when a network is loaded and enabled, otherwise to the classical evaluation. Either can be selected at runtime via the `Use NNUE` UCI option.

4.1 Classical (hand-crafted) evaluation

A **tapered** evaluation blending middlegame and endgame scores by a material "phase": published **PeSTO** piece-square tables, plus piece mobility, passed / isolated / doubled pawn terms, the bishop pair, rooks on open and semi-open files, and a tempo bonus. This evaluation alone, with the search above, defeated Stockfish 17.1 limited to 2600 Elo by 6–4 — above the human grandmaster threshold — and served as the **first teacher** for the neural network.

4.2 NNUE — efficiently updatable neural evaluation

The production network is a **(768 → 128) × 2 perspective architecture**:

- **Inputs (768).** One feature per (colour, piece type, square), evaluated from each side's point of view; the black perspective flips colour and mirrors the rank.
- **Accumulator (128 × 2).** The first layer's output is a sum of weight columns, one per active feature. Because a move changes only a few features, the accumulator is updated **incrementally** inside make/unmake — add the columns that appeared, subtract those that vanished, with exact inverse deltas on unmake. This mirrors the Zobrist key's incrementality (a vector instead of a scalar) and was verified bit-exact against full recomputation across castling, promotions, en passant, and king moves.
- **Output.** Both perspectives (side-to-move first) pass through a clipped-ReLU and a single linear layer to a centipawn score.
- **Quantization.** Weights are quantized to **int16** (input scale QA = 255, output scale QB = 64, eval scale = 400) and inference runs in integer arithmetic with int32 accumulation, so the whole forward pass is a few hundred integer multiply-adds per leaf.

The naive recompute-per-leaf version ran at ~280K nps; the incremental, vectorized version restored this to ~720K nps. The network file format (`NARCNET1`) carries the hidden size so the loader can reject mismatched nets.

4.3 The HalfKA experiment

A second architecture, **HalfKA** (king-bucketed), was built and tested: every piece-square input is replicated across **8 king buckets** (6,144 inputs per perspective) with horizontal mirroring, so the network can learn king-relative knowledge such as king safety. Despite fitting the training data substantially better (lower validation loss on identical data), it **lost decisively in play** (see §8). The accumulator for this architecture uses snapshot-based undo because a king move can change its perspective's bucket or mirror and force a full perspective rebuild. The experiment is preserved in the repository as a documented negative result.

---

5. Parallel search (Lazy SMP)

NARC parallelizes with **Lazy SMP**: rather than partitioning the tree (which is hard, since alpha-beta bounds are inherently sequential), *N* threads search the **same root** on independent board copies, sharing only the transposition table. The threads desynchronize naturally — helpers are offset to start one ply deeper — so they explore different subtrees, and the shared TT is the communication channel through which a bound or move found by one thread instantly helps the others.

The implementation moved all search state into a **per-thread** `SearchData` (histories, killers, principal variation, repetition keys), made the TT race-tolerant (§3.2), and added a thread pool driven by the new `Threads` UCI option (1–256). The main thread owns time management and output; helpers neither print nor manage the clock. Data generation is pinned to a single thread for determinism.

Correctness was established two ways. First, the single-threaded path is **bit-for-bit identical** to the previous release — the same position searched to the same depth produces exactly 286,029 nodes and the same principal variation — proving the large refactor changed nothing about the search itself. Second, multithreaded self-play was refereed move-by-move by an independent library (`python-chess`) with zero illegal moves, confirming the parallel code is race-safe for legality. Four threads deliver roughly 3× the node rate on a two-core (plus hyperthreading) laptop.

---

6. Training methodology

NARC's neural network is trained entirely on data the engine generates by playing itself. The pipeline has four stages: data generation, supervised training, deployment, and — the key to ongoing gains — generational iteration.

6.1 Data generation

A `datagen` command is **built into the engine itself**. It plays self-play games from randomly perturbed openings at a fixed node budget per move, applies light opening-balance filtering and win/draw adjudication, and logs one record per quiet position:

```
<FEN> | <search score, White's perspective, cp> | <game result, White POV>
```

Crucially, the **label is the engine's *search* score**, not its static evaluation — search is stronger than the raw evaluation, so the network learns to approximate searched judgments, effectively distilling a slice of search depth into the static eval.

6.2 Network training

Training is done in PyTorch. Each position's target blends the search score and the game result through a logistic function:

```
target = λ · sigmoid(score / 400) + (1 − λ) · result          (λ = 0.7)
```

The network is optimized (Adam, MSE on the win-probability prediction) over the position set, then **quantized** to the int16 format the engine loads. A separate verification script cross-checks the Python feature mapping against the C++ engine **bit-exactly** before any training run, eliminating the most dangerous failure mode (a silent feature mismatch that would make the trained weights meaningless).

6.3 Cloud pipeline

Because data generation is CPU-bound and embarrassingly parallel while training is GPU-bound, the two are split across cloud resources (Modal):

- **Data generation** fans out across many one-core CPU containers (each running the engine's `datagen`), writing shards to a shared volume — hundreds of CPU cores in parallel.
- **Training** runs on a single **A10G GPU**, reading all shards and writing the quantized network back to the volume (≈7–9 seconds per epoch on millions of positions).

A full generation (multi-million positions generated and a network trained) costs on the order of a few US dollars and roughly an hour of wall-clock time. The entire pipeline is scripted and documented for reproduction.

6.4 Generational self-play

The central training idea is a **teacher progression**. Generation 1 was labeled by the classical evaluation. Once that network surpassed the classical eval, it became the teacher for generation 2; generation 2 then taught generation 3; and so on. Each generation learns from a *stronger* search than the last, so a faithful student can *exceed* its predecessor rather than merely imitate it — the same self-improvement principle as AlphaZero, applied here as discrete supervised distillation on a classical alpha-beta engine rather than continuous reinforcement learning with tree search.

```
classical eval  →  gen-1 net  →  gen-2 net  →  gen-3 net  →  …
   (teacher)        (teacher)     (teacher)
```

---

7. Testing and validation methodology

The testing discipline is itself a significant part of the engineering, and it produced two hard-won methodological lessons.

- **Move generation** is validated by perft against known-correct node counts on the standard adversarial positions.
- **Game legality** under every change (and especially under multithreading) is validated by refereeing self-play through `python-chess`, which rejects any illegal move.
- **Strength** is measured only by **engine-vs-engine matches**, never by proxy.

Lesson 1 — validation loss does not predict playing strength. This inverted in *both* directions during the project: a network with mediocre training loss won its match convincingly, while a different network with markedly *lower* loss lost by a wide margin. Every strength decision was therefore made on game results, not on the loss curve.

Lesson 2 — gate at realistic time controls. Early gating used a fixed milliseconds-per-move budget, which structurally handicaps NNUE (it pays NNUE's lower node rate without giving the evaluation quality time to matter at depth). Switching to a **real game clock** (base time plus increment, with the engine's own time management active) reversed an apparent failure into a clear win, and became the standard for all subsequent gates.

A dedicated match harness implements both fixed-movetime and game-clock modes and reports the score with an Elo estimate; an independent referee validates every move.

---

8. Empirical results — nine generations

The table summarizes the measured outcome of every major version. Strength figures are from 40-game matches at an 8-second-plus-0.08 game clock unless noted; the classical-vs-Stockfish result is 10 games at 200 ms/move.

Version	Change	Result	Verdict
1.0 / 1.1 (classical)	Tapered hand-crafted eval + full search	**6–4 vs Stockfish @2600 Elo**	Above GM level; shipped
2.1 (gen-1 NNUE)	Net trained on 4.4M classical-labeled positions	Beat the classical eval at real time controls (blitz 7.5–2.5)	Shipped
2.2 (gen-2 NNUE)	Net trained on v2.1's *search* (generational)	**+61 Elo vs v2.1**	**Shipped — large, real gain**
2.3 (gen-3 NNUE)	Net trained on v2.2's search	+17 Elo vs v2.2 (within noise)	Plateau — not shipped
2.4 (wider flat, H=256)	Doubled hidden width	−35 Elo vs v2.2 (within noise)	Capacity not the limit — not shipped
3.0 (HalfKA)	King-bucketed features, classical-labeled	−147 Elo vs v2.1	Rejected
3.1 (HalfKA)	King-bucketed, strong teacher + 10.8M positions	−168 Elo vs v2.2	Rejected — architecture deficit
2.5 (Lazy SMP)	Multithreaded search (net unchanged)	**+137 Elo at 2 threads vs 1** (only 3 losses / 40)	**Shipped — largest gain**

Three conclusions follow directly from these numbers:

1. **The generational loop works, but saturates.** Gen-2 gained +61; gen-3 only +17. The compact 128-wide network reached the ceiling of what it could absorb from an increasingly sophisticated teacher.
2. **Capacity was not the bottleneck.** Neither a wider flat network (−35) nor the higher-capacity HalfKA architecture (−147, −168) improved on v2.2 — both fit the data better yet played worse, a textbook case of over-fitting a teacher whose own knowledge is the limit. This is a clean, documented negative result.
3. **Parallel search was the decisive lever.** With evaluation at its ceiling, the +137-Elo SMP gain — the project's largest — came from the search side, and scales further on machines with more cores.

The honest arc: one large evaluation gain (the generational jump), a rigorously established plateau, and a large search-side gain. Negative results were retained and reported rather than hidden.

---

9. Distinguishing characteristics and novelty

It is important to be precise here. **NARC does not introduce a novel search algorithm or a novel network architecture.** Every technique it uses — alpha-beta, magic bitboards, NNUE, Lazy SMP, generational distillation — is established work, much of it documented on the Chess Programming Wiki and pioneered by engines such as Stockfish and the AlphaZero/Leela lineage. Claiming algorithmic novelty would be inaccurate.

What distinguishes NARC is **integration, self-containment, and methodological transparency**:

1. **Fully self-bootstrapped, with zero borrowed knowledge.** The dominant way hobby engines obtain a strong NNUE is to reuse Stockfish's published network. NARC does not: its network is trained from random initialization on the engine's *own* self-play, its magic constants are searched rather than copied, and no external engine, book, tablebase, or dataset is used anywhere. The 3.6 MB binary's entire evaluation ancestry is its own games. (Few engines — Seer being a notable exception — can make this claim end to end.)
2. **The complete self-improvement loop in one small artifact.** The data generator is a command *inside the engine*. The full cycle — play, label by search, train, quantize, embed back into the executable — closes with no external dependency, and is small enough (~2,000 lines plus a compact training script) to be read end to end. Most engines that train their own nets split this across separate tools and repositories.
3. **A reproducible, low-cost cloud pipeline.** The generational loop is packaged as a documented Modal pipeline (CPU fan-out for datagen, a single A10G for training) that produces a new generation for a few dollars in about an hour, making self-play NNUE accessible without a GPU cluster.
4. **Documented empirical rigor, including negative results.** The project gated every change in real matches, discovered and corrected its own measurement methodology (the fixed-movetime lesson), and *kept and published the experiments that failed* (two HalfKA attempts, the wider net, the gen-3 plateau). This transparency about what did not work is unusual in engine write-ups and is, arguably, the project's most distinctive contribution.
5. **Single-file deployment with verified-identical fallback.** The network is embedded in the executable (no companion files), an external net can override it, and the engine falls back to the classical evaluation if no net is present. The multithreading refactor was proven to leave single-threaded play bit-for-bit unchanged.

In short, NARC's contribution is not a new idea but a **faithful, transparent, and fully self-contained realization of the modern engine recipe at small scale** — "Stockfish in miniature," built and documented in the open, including the parts that did not work.

---

10. Engineering and deployment

- **Language / standard:** C++20, single translation unit (everything `inline`), compiled with `-O3 -march=native -flto -static`.
- **Toolchain:** GCC (MSYS2 UCRT64 on Windows); `-pthread` for SMP; `windres` for the embedded icon and version resource.
- **Protocol:** full UCI, on a dedicated search thread, with options for `Hash`, `Threads`, `Move Overhead`, `Use NNUE`, and `EvalFile`. Console helpers (`perft`, `eval`, board display, `datagen`) aid development.
- **Deployment:** a single static `.exe` with the network embedded; no runtime dependencies. Verified to load the built-in net from any working directory.
- **Distribution:** open source under the GNU GPL v3.0-or-later; published with prebuilt binaries and the full training pipeline.

---

11. Limitations and future work

- **Strength ceiling of the current network.** The 128-wide flat network has plateaued; capacity experiments showed that simply enlarging it (wider or king-bucketed) does not help with the current label depth. The remaining untested lever is **deeper search labels** (higher node budget per training position), which would sharpen the teacher signal the network learns from.
- **HalfKA underperformance.** The king-bucketed architecture loses despite higher representational capacity, likely due to evaluation discontinuities at bucket boundaries and over-fitting a fixed teacher; making it competitive would require a substantially stronger teacher and far more data, or architectural smoothing.
- **No endgame tablebases or opening book.** Both are standard, orthogonal gains not yet implemented.
- **Single-machine pipeline.** The cloud pipeline is budget-oriented; reaching the data scale at which higher-capacity networks pay off (tens of millions to billions of positions) would require a larger compute commitment.

---

12. Conclusion

NARC demonstrates that the complete modern chess-engine recipe — magic bitboards, a fully-featured alpha-beta search, an incrementally-updated NNUE evaluation trained by generational self-play, and Lazy SMP parallelism — can be implemented faithfully and transparently from scratch in a compact, self-contained codebase, with no borrowed code, data, or networks. Its measured results validate the approach (a +61-Elo generational evaluation gain and a +137-Elo parallelism gain on top of a Stockfish-2600-beating classical baseline) while its rigorously documented negative results map the limits of that approach. NARC's distinction lies not in algorithmic novelty but in being an open, reproducible, and honest end-to-end realization of how a strong engine is actually built — including the experiments that failed.

Appendix A — Key parameters

Parameter	Value
Move encoding	16-bit (from, to, promo, flag)
Magic tables	~107 KB (rook + bishop), searched at startup
TT default size	128 MB; race-tolerant XOR entries
NNUE (production)	(768 → 128) × 2 perspectives, int16
NNUE quantization	QA = 255, QB = 64, eval scale = 400
HalfKA (experimental)	8 king buckets × 768 → 256, mirrored
Training target	0.7 · sigmoid(score/400) + 0.3 · result
Threads	1–256 (UCI option); default 1
Node rate	~1.0M nps (1 thread) / ~3.0M nps (4 threads)

Appendix B — Source layout

```
src/
  types.h         bitboards, magic attacks, Zobrist, move encoding
  position.h      board, make/unmake (+ NNUE accumulator), movegen, SEE, perft
  eval.h          tapered classical evaluation
  nnue_params.h   NNUE weights + accumulator primitives
  nnue.h          NNUE load and inference; evaluation dispatch
  search.h        transposition table, search, Lazy SMP, time control
  main.cpp        UCI protocol, datagen command, thread pool driver
src3/             parallel source tree for the HalfKA (v3) architecture
test/             python-chess referee, match harness, NNUE trainer, net embedder
modal/            cloud pipeline (CPU datagen fan-out + A10G GPU training)
```

This report describes the engine and pipeline as implemented. For a conceptual overview see `HOW_IT_WORKS.md`; for a code-level walkthrough and algorithm theory see `CODE_AND_THEORY.md`; for build and usage see `README.md`.